Algorithmie

SAÉ 3.02 - Labyrinthe

Développement efficace

Étudiants :

Victor Bredelle
Antonin Marouze
Romain Harlaut
Angèl Zheng
Baptiste Lavogiez

Enseignant :

Mme Everaere

15 novembre 2025

Table des matières

1 Introduction **3**

2 Investissement dans le projet **4**

3 Structures de données pour les labyrinthes **6**
3.1 Labyrinthes aléatoires 6
3.2 Labyrinthes parfaits 6

4 Algorithmes de génération **7**
4.1 Distance entrée / sortie 7
4.1.1 Algorithme utilisé 7
4.1.2 Pseudo-code 8
4.1.3 Structures de données utilisées 9
4.1.4 Pertinence 10
4.2 Labyrinthes parfaits : Version TP 10
4.2.1 Algorithme utilisé 10
4.2.2 Pseudo-code 11
4.2.3 Structures de données utilisées 11
4.2.4 Pertinence 11
4.3 Labyrinthes parfaits : Version originale - Fusion 11
4.3.1 Algorithme utilisé 11
4.3.2 Pseudo-code 12
4.3.3 Structures de données utilisées 12
4.3.4 Pertinence 12
4.4 Labyrinthes aléatoires 13
4.4.1 Algorithme utilisé 13
4.4.2 Pseudo-code 13
4.4.3 Structures de données utilisées 13
4.4.4 Pertinence 14

5 Efficacité **15**
5.1 Méthode 15
5.1.1 Test de performance 15
5.1.2 Modélisation 16
5.2 Résultats 17

5.2.1 Labyrinthes aléatoires 17
5.2.2 Labyrinthes parfaits 17
5.2.3 Comparaison parfaits / aléatoires 19
5.3 Conclusion 19

6 Complément et Bilan **20**
6.1 Distance entrée / sortie 20
6.2 Placement des entités 20
6.3 Déplacement des monstres 20

7 Conclusion **21**

1 Introduction

Ce rapport présente la partie algorithmique du projet **SAÉ S3.02 - Labyrinthe**, une application permettant la génération de labyrinthes variés. L'objectif est de décrire notre solution et de démontrer sa pertinence, guidée par l'efficacité d'exécution et les exigences du sujet.

2 Investissement dans le projet

Groupe G3.3

- Victor Bredelle
- Antonin Marouze
- Romain Harlaut
- Angèl Zheng
- Baptiste Lavogiez

Nom	Victor	Antonin	Romain	Angèl	Baptiste	Total
Part	⟨Data 2.1⟩	⟨Data 2.2⟩	⟨Data 2.3⟩	⟨Data 2.4⟩	⟨Data 2.5⟩	100%

Table 2.1 – Répartition de l'investissement

1. Victor

2. Antonin

3. Romain

4. Angèl

Algorithme de déplacement de monstre.

5. Baptiste

- Algorithme distance entrée / sortie.
- Algorithme pour les labyrinthes aléatoires.
- Benchmarks de performance des labyrinthes et modélisation.

Le groupe

Chaque section est écrite par la personne l'ayant réalisée, et éventuellement relue par le groupe.

3 Structures de données pour les labyrinthes

3.1 Labyrinthes aléatoires

3.2 Labyrinthes parfaits

4 Algorithmes de génération

4.1 Distance entrée / sortie

Cet algorithme sera utilisé dans les algorithmes à venir.

Le placement d'une distance entrée / sortie sera ici considéré comme un algorithme à part. Le respect strict de la distance nécessite en effet de réaliser un parcours intelligent vers la sortie, depuis l'entrée, la distance les séparant.

4.1.1 Algorithme utilisé

Une approche par niveaux sera utilisée ici. L'algorithme sera un **parcours en largeur**.

4.1.2 Pseudo-code

On note :

- d : la distance cible entre l'entrée et la sortie.

pseudocode

```
créer une position aléatoire entrée dans le labyrinthe

créer une file vide
créer une map distances vide
créer une liste positionsResultat vide
ajouter entrée à la file
ajouter (entrée, 0) à distances
niveauActuel = 0
distanceReelle = 0

tant que la file n'est pas vide :
    tailleNiveau = taille de la file

    si niveauActuel > d :
        arrêter le parcours

    si niveauActuel = d :
        pour i de 0 à tailleNiveau :
            position = retirer premier élément de la file
            ajouter position à positionsResultat
            distanceReelle = niveauActuel
            arrêter le parcours

    pour i de 0 à tailleNiveau :
    positionCourante = retirer premier élément de la file
    pour chaque direction {haut, bas, gauche, droite} :
        prochaine = calculer position adjacente selon
           direction
        si prochaine valide et prochaine non visitée et pas
            de mur :
```

```
                ajouter (prochaine, niveauActuel + 1) à distances
                ajouter prochaine à la file

        distanceReelle = niveauActuel
        niveauActuel = niveauActuel + 1

    si positionsResultat est vide :
        pour chaque (position, distance) dans distances :
        si distance = distanceReelle :
        ajouter position à positionsResultat

    sortie = choisir aléatoirement une position dans positionsResultat
    définir entrée dans le labyrinthe
    définir sortie dans le labyrinthe
    mettre à jour la distance effective avec distanceReelle
```

Notre solution permet alors un strict respect de la distance entrée / sortie, et surtout, un calcul de la distance maximale possible afin de ramener la distance voulue à celle maximale si celle voulue était impossible dans ce labyrinthe. Cette notion est **importante pour la suite**.

La mise à jour de la distance effective permet, entre autres, de prévenir la structure de labyrinthes que la distance voulue a changé, permettant un retour utilisateur.

4.1.3 Structures de données utilisées

Pendant l'exécution, l'algorithme utilise principalement une file et un dictionnaire.

- La file est indispensable puisqu'il s'agit d'un parcours en largeur. Elle nous permet d'explorer plusieurs niveaux à la fois (logique plus efficace).
- Le dictionnaire sert à associer une position à sa distance. Elle permet

Quant au retour, l'algorithme utilise un objet "distanceResultat" contenant :

- Une liste de positions.

- La distance effective.

Puis, il choisit aléatoirement une position parmi la liste. Sera attribuée au labyrinthe la distance effective, afin d'avertir de l'impossibilité de celle initialement voulue.

4.1.4 Pertinence

La motivation principale quant au choix des structures est l'efficacité. La structure choisie dans ce cas est minimale et comporte uniquement ce dont nous avons besoin pour déterminer une position aléatoire respectant la distance demandée (ou maximum si impossible).

4.2 Labyrinthes parfaits : Version TP

4.2.1 Algorithme utilisé

4.2.2 Pseudo-code

pseudocode

```
toto titi
tant que toto
faire tata
```

4.2.3 Structures de données utilisées

4.2.4 Pertinence

4.3 Labyrinthes parfaits : Version originale - Fusion

4.3.1 Algorithme utilisé

4.3.2 Pseudo-code

pseudocode

```
toto titi
tant que toto
faire tata
```

4.3.3 Structures de données utilisées

4.3.4 Pertinence

4.4 Labyrinthes aléatoires

L'algorithme de génération de labyrinthe aléatoire héritant de l'algorithme parfait, cette sous-section est détaillée après la précédente.

L'objectif sera de partir d'un labyrinthe parfait et de le rendre imparfait. Cette solution permet un respect des propriétés "parfaites" par héritage.

Le chemin entre l'entrée et la sortie est, par définition, garanti, puisqu'un labyrinthe parfait est un graphe connexe.

La propriété connexe demeure inchangée tant qu'aucun mur n'est ajouté.

Cet algorithme n'ajoute aucun mur, il ne fait qu'en enlever.

Puisqu'un labyrinthe aléatoire est ici un graphe connexe, il existe toujours un chemin entre l'entrée et la sortie.

4.4.1 Algorithme utilisé

L'algorithme consiste à, pour un pourcentage de murs noté p, parcourir les murs, et, en enlever par un tirage de probabilité (1 - p).

Concernant la distance entrée / sortie, puisque sans paramètre dans le sujet, elle sera ramenée au maximum possible du labyrinthe, comme prévu par l'algorithme de placement.

4.4.2 Pseudo-code

On note :

- m : un labyrinthe parfait
- p : le pourcentage de murs souhaité

pseudocode

```
pour y allant de 0 à hauteur - 1
  pour x allant de 0 à largeur
    si mur horizontal à [y][x]
      si valeur aléatoire entre (0,1) inférieure à (1 - p)
        enlever mur horizontal à [y][x]
pour y allant de 0 à largeur - 1
  pour x allant de 0 à hauteur
    si mur vertical à [y][x]
```

```
        si valeur aléatoire entre (0,1) inférieure à (1 - p)
          enlever mur vertical à [y][x]
```

4.4.3 Structures de données utilisées

Les structures utilisées sont identiques à celle de l'algorithme parfait utlisé, que ce soit celui de *TP ou Fusion*.

4.4.4 Pertinence

La pertinence est donc identique.

5 Efficacité

5.1 Méthode

La présente sous-section expose notre méthodologie de manière théorique, divisée en deux parties distinctes.

1. Test de performance
2. Modélisation

5.1.1 Test de performance

On note :

- a : l'algorithme du labyrinthe,
- n : la taille initiale du labyrinthe,
- e : l'écart entre deux tests successifs,
- f : la taille maximale (fin du test).

$$\forall n > 2,\ n \in \mathbb{N}, \forall e \in \mathbb{N} :$$

pseudocode

```
créer un fichier csv de nom "a_n_e_f"
tant que n < f :
    t1 = mesurer le temps actuel
    générer un labyrinthe d'algorithme (a) de taille n (
        largeur = n, hauteur = 2*n, distance = n)
    t2 = mesurer le temps actuel
    écrire sur la ligne actuelle (n, t2 - t1)
    n = n + e
    passer à la ligne suivante
```

Information

Lorsque la distance entrée est de 0, l'algorithme de distance entrée/sortie ne s'exécute pas. Cela permet d'observer le coût brut de la génération ou de la distance entrée/sortie, plutôt qu'une limitation à une combinaison des deux. Cette idée sera appliquée dans certaines analyses.

Taille (n)	Temps de génération (ms)
10	12
15	18
20	25
25	30
30	38
35	45
⋮	⋮
495	1200
500	1250

Table 5.1 – Exemple de sortie pour $a = \text{STANDARDLARGEUR}, n = 10, e = 5, f = 500$

Implémentation Java

5.1.2 Modélisation

Ces fichiers sont modélisés, seul ou à plusieurs, par un script (python, matplotlib).

Seront uniquement présentés en résultat ces modélisations graphiques.

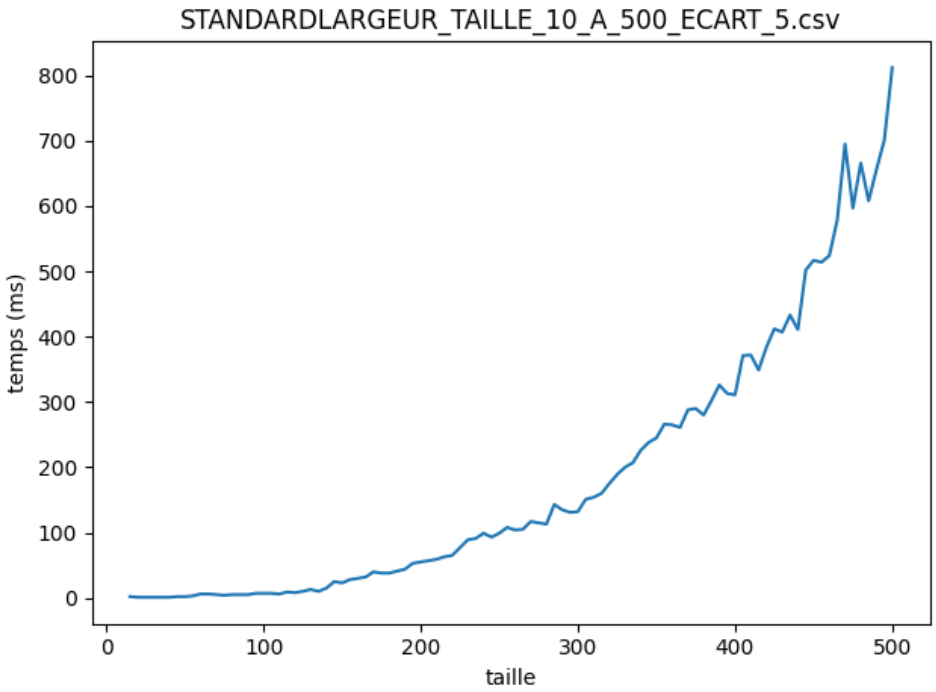

Figure 5.1 – Exemple de sortie pour $a = \text{STANDARDLARGEUR}, n = 10, e = 5, f = 500$

5.2 Résultats

5.2.1 Labyrinthes aléatoires

5.2.2 Labyrinthes parfaits

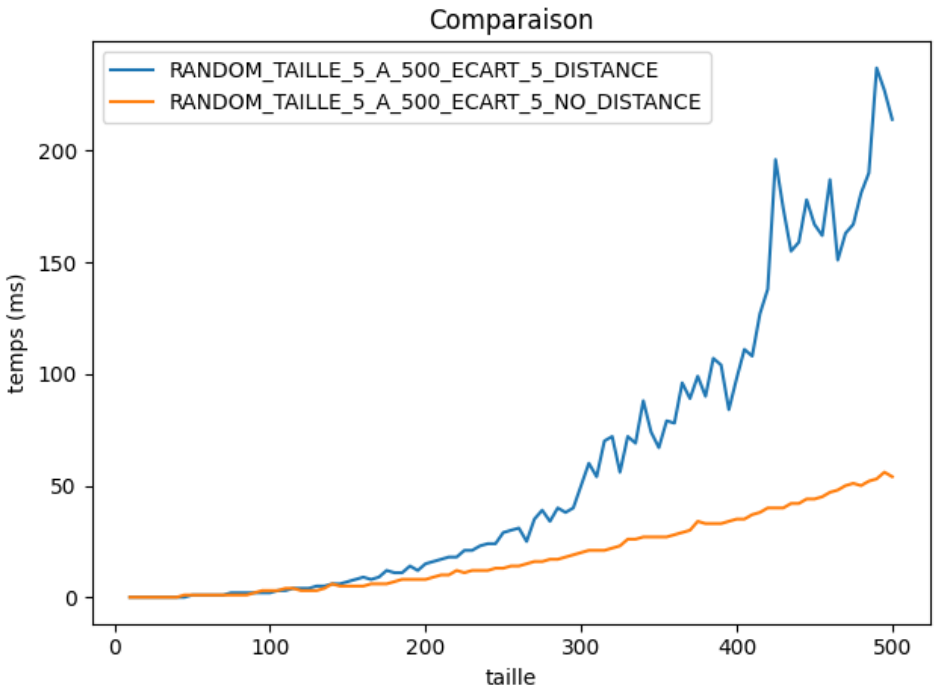

FIGURE 5.2 – $a = \text{RANDOM}, n = 5, e = 5, f = 500$

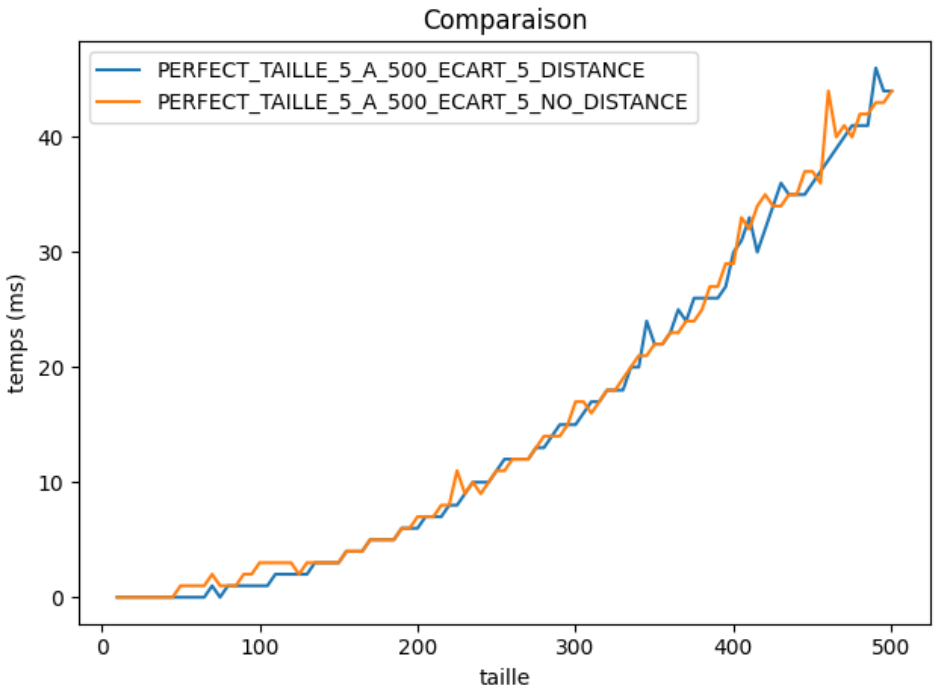

FIGURE 5.3 – $a = \text{PERFECT}, n = 5, e = 5, f = 500$

Algorithme vu en TP

5.2.3 Comparaison parfaits / aléatoires

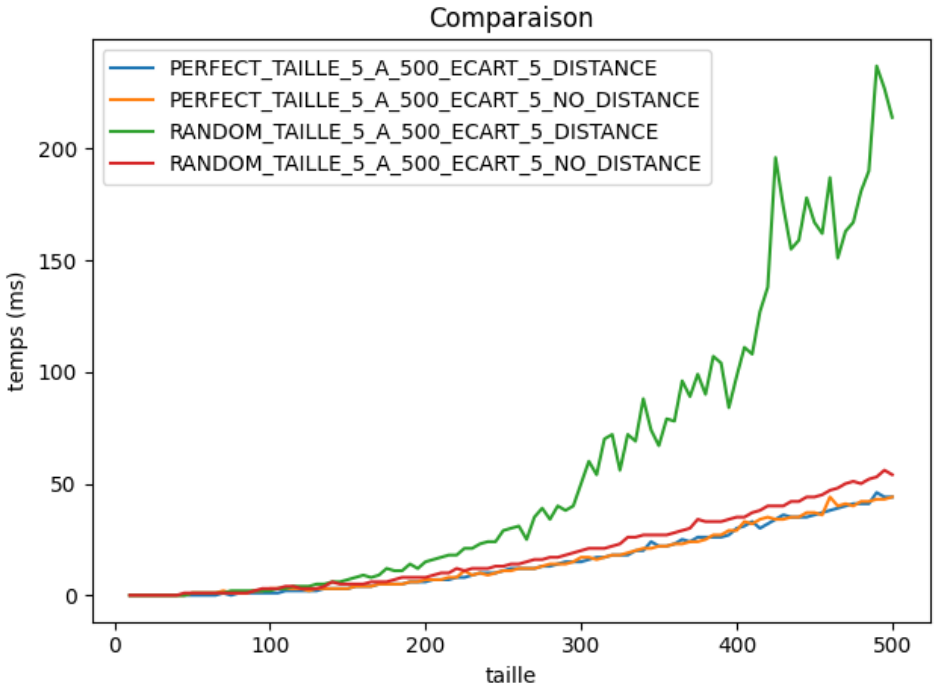

FIGURE 5.4 – Enter image caption...

Algorithme original : Fusion

5.3 Conclusion

6 Complément et Bilan

6.1 Distance entrée / sortie

Notre solution actuelle est optimisée, mais elle n'est apparue qu'après plusieurs tentatives. En effet, notre première solution consistait à :

- Faire un dictionnaire (clé : Position, valeur : Distance) de toutes les positions depuis la position en paramètre par parcours en largeur.
- Puis, choisir toutes les positions satisfaisant

En réalité, cette approche est inefficace puisqu'elle implique de parcourir absolument tout le labyrinthe. Elle avait été imaginée pour provisionner une solution simple au problème du maximum de la distance entrée / sortie.

Les performances de l'algorithme parfait ont été multipliées par 10 après application de la nouvelle solution.

6.2 Placement des entités

Notre projet comporte des **entités** (*joueurs, monstres, sorties...*) qui doivent être placées initialement selon des stratégies. Le problème étant très similaire à celui du placement de l'entrée et de la sortie, un algorithme de la sorte a été implémenté pour le calcul de leurs positions de départ.

La différence étant que le placement vérifie l'absence d'entités sur cette case. L'algorithme enlève alors les positions occupées pour le niveau exploré. Si le niveau demandé ne comporte aucune position valide, alors nous explorons un niveau supérieur.

6.3 Déplacement des monstres

Section Angèl :

7 Conclusion